

AppChat

AppChat เป็นแอปพลิเคชันที่ใช้สื่อสาร (Network Chat Application) มีการสร้าง Network Chat Protocol (NCP) มาใช้ โดยรูปแบบการสื่อสารเป็นแบบ Client-Server และใช้ Transport Layer เป็น TCP

วัตถุประสงค์

พัฒนาโปรแกรมสนทนาแบบเครือข่ายระหว่างผู้ใช้หลายคน โดยแอปพลิเคชันมีความสามารถได้ดังนี้

- ให้ Client สามารถ Login เข้าสู่ระบบ
- แสดงรายชื่อผู้ใช้งานที่กำลัง Online
- ส่งข้อความแบบระบุผู้รับ
- ส่งข้อความแบบ Broadcast ให้ผู้ใช้อื่น
- รองรับการรับข้อความแบบ Real-time
- แสดงสถานะการทำงานของ Protocol ให้ตรวจสอบได้

ลักษณะของ Application

Client-Server Architecture

Client จะไม่ได้ส่งข้อความถึงกัน มี Server เป็นตัวกลาง โดย Server ทำหน้าที่

- รับ Request
- ตรวจสอบผู้ใช้
- จัดการรายชื่อ Online
- ส่งข้อความไปยังผู้รับ
- ส่ง Response กลับ Client

Multi-Client

รองรับ Client หลายเครื่อง/หลายหน้าต่างเชื่อมต่อ Server พร้อมกัน

Real-time Messaging

เมื่อผู้ส่งส่งข้อความ Server จะส่งข้อความต่อไปยังผู้รับทันที

Message Identification

แต่ละข้อความมี Message ID เช่น MSG-0001, MSG-0002, MSG-0003

Status Code

Protocol มี Status Code และ Status Phrase เพื่อบอกผลลัพธ์ของ Request เช่น 200 OK, 400 BAD_REQUEST, 404 NOT_FOUND, 409 CONFLICT

Transport Layer Service Model

การพัฒนาแอปพลิเคชันนี้ใช้ TCP (Transmission Control Protocol) เนื่องจากมีความน่าเชื่อถือ สามารถส่งข้อความให้ถึงปลายทางได้อย่างถูกต้อง ตามลำดับที่ควรจะเป็น และส่งข้อมูลถึงที่หมายครบทุก Packet

และถึงแม้ว่า UDP มีข้อดีด้าน Overhead และความเร็ว แต่ไม่มีการรับประกันการส่งข้อมูลและลำดับของ Packet ซึ่งไม่เหมาะกับ Chat Application ที่ต้องการให้ข้อความถึงผู้รับอย่างถูกต้องและครบถ้วน ดังนั้นจึงเลือกใช้ TCP

Application-Layer Protocol

NCP เป็น Application-Layer Protocol ที่ถูกออกแบบมาเพื่อใช้กับ AppChat โดยเฉพาะ

Protocol Format

Request จะมีรูปแบบเป็น COMMAND [PARAMETER] เช่น

- LOGIN User
- LIST
- SEND Desti_User Hello

Request Messages

คำสั่ง Request มีดังนี้

- LOGIN [*User*]

ใช้สำหรับระบุตัวตนของ Client ว่าผู้ใช้งานคือใคร ก่อนที่จะสามารถใช้คำสั่งอื่น ๆ เช่น LIST, SEND และ SENDALL

เมื่อ Server ได้รับคำสั่ง LOGIN จะตรวจสอบว่า Username นั้นกำลังถูกใช้งานอยู่หรือไม่

```
Enter username: Apple
[20:30:31] [CLIENT] [SEND]
    LOGIN Apple

[20:30:31] [CLIENT] [RECEIVE RESPONSE]
    200 OK LOGIN_SUCCESS Apple
```

- LIST

ใช้สำหรับขอรายชื่อผู้ใช้ที่กำลัง Online จาก Server

```
LIST
[20:32:13] [CLIENT] [SEND]
    LIST

[20:32:13] [CLIENT] [RECEIVE RESPONSE]
    200 OK USER_LIST Apple,Rum
```

คำสั่งนี้ช่วยให้ผู้ใช้ทราบว่า สามารถส่งข้อความหาใครได้บ้าง โดยในระบบของเรา Server จะสร้างรายชื่อจาก online_users ซึ่งเก็บ Username และ Socket ของผู้ใช้ที่ Login อยู่

- SEND [User] [Message]

มีหน้าที่ส่งข้อความไปยังผู้ใช้ที่กำหนด หนึ่งต่อหนึ่ง

```
> SEND Rum Hi  
  
[20:39:42] [CLIENT] [SEND]  
SEND Rum Hi  
  
[20:39:42] [CLIENT] [RECEIVE RESPONSE]  
200 OK MESSAGE_SENT MSG-0002
```

ผู้ส่ง

```
[20:39:42] [CLIENT] [INCOMING]  
MESSAGE MSG-0002 MESSAGE Apple Hi
```

ผู้รับ

ตัวอย่างในภาพ Apple ต้องการส่ง Hi ให้ Rum: SEND Rum Hi Server จะตรวจสอบก่อนว่า:

1. ผู้ส่ง Login แล้วหรือไม่
2. มีผู้ใช้ Rum อยู่หรือไม่
3. สามารถส่งข้อความไปยัง Socket ของ Rum ได้หรือไม่

- SENDALL [Message]

มีหน้าที่ส่งข้อความไปยังทุกคนที่ออนไลน์อยู่

```
> SENDALL Wassap  
  
[20:44:34] [CLIENT] [SEND]  
SENDALL Wassap  
  
[20:44:34] [CLIENT] [RECEIVE RESPONSE]  
200 OK BROADCAST_SENT MSG-0003
```

ผู้ส่ง

```
[20:44:34] [CLIENT] [INCOMING]  
MESSAGE MSG-0003 Apple Wassap
```

ผู้รับ

- QUIT

ใช้สำหรับให้ Client ออกจากระบบอย่างถูกต้อง

```
quit  
  
[20:46:24] [CLIENT] [SEND]  
QUIT  
  
[20:46:24] [CLIENT] [RECEIVE RESPONSE]  
200 OK BYE  
  
[PERFORMANCE] Response Time: 1.26 ms  
  
[STATUS] You are now offline.
```

Response Messages

Response Messages มีดังนี้

Status Code	Status Phrase	ความหมาย
200	OK	Request สำเร็จ
400	BAD_REQUEST	Request ไม่ถูกต้อง
401	UNAUTHORIZED	ยังไม่ได้ระบุตัวตน
404	NOT_FOUND	ไม่พบผู้ใช้
409	CONFLICT	เกิดความขัดแย้ง เช่น Username ซ้ำ
500	SERVER_ERROR	เซิร์ฟเวอร์ล่ม

ตัวอย่าง

```
Enter username: Apple

[20:52:42] [CLIENT] [SEND]
LOGIN Apple

[20:52:42] [CLIENT] [RECEIVE RESPONSE]
409 CONFLICT USERNAME_TAKEN
```

409 Username ซ้ำ

```
> SEND Som Hi

[20:54:21] [CLIENT] [SEND]
SEND Som Hi

[20:54:21] [CLIENT] [RECEIVE RESPONSE]
404 NOT_FOUND USER_NOT_FOUND
```

404 ไม่พบผู้ใช้

```
[17:26:10] [CLIENT] [RECEIVE]
200 OK LOGIN_SUCCESS Apple
```

200 คำขอสำเร็จ

ข้อจำกัดของระบบ

ระบบ AppChat ที่พัฒนาขึ้นเป็นโปรแกรม Chat แบบ Client-Server ซึ่งใช้ TCP เป็น Transport Layer และใช้ NCP (Network Chat Protocol) เป็น Application-Layer Protocol สำหรับกำหนดรูปแบบการสื่อสารระหว่าง Client และ Server อย่างไรก็ตาม ระบบยังมีข้อจำกัดบางประการ ดังนี้

รองรับเฉพาะการสื่อสารแบบข้อความ

ระบบในปัจจุบันรองรับเฉพาะการส่งข้อความตัวอักษร (Text Message) ผ่านคำสั่ง SEND และ SENDALL ยังไม่รองรับการส่งไฟล์ รูปภาพ เสียง หรือวิดีโอ

ข้อมูลผู้ใช้งานจัดเก็บเฉพาะในหน่วยความจำ

รายชื่อผู้ใช้งานที่กำลัง Online ถูกจัดเก็บไว้ในตัวแปร online_users ของ Server ดังนั้นเมื่อ Server ถูกปิดหรือ Restart ข้อมูลผู้ใช้งานที่กำลัง Online จะถูกล้างออกทั้งหมด และระบบยังไม่มีฐานข้อมูลสำหรับจัดเก็บบัญชีผู้ใช้งานอย่างถาวร

Username ต้องไม่ซ้ำกันในเวลาที่ Online

ระบบกำหนดให้ Username ของผู้ใช้งานที่กำลัง Online ต้องไม่ซ้ำกัน หากผู้ใช้งานพยายาม Login ด้วย Username ที่มีผู้ใช้งานอยู่แล้ว Server จะส่ง Status Code “409 CONFLICT USERNAME_TAKEN” ดังนั้น Client จะต้องเลือก Username ใหม่ก่อนจึงจะสามารถเข้าสู่ระบบได้

ไม่มีระบบยืนยันตัวตนแบบรหัสผ่าน

การ Login ของระบบใช้เพียง Username เพื่อระบุตัวตน ยังไม่มีระบบ Password หรือระบบ Authentication ที่มีความปลอดภัยในระดับสูง ดังนั้นระบบนี้เหมาะสำหรับการสาธิตการทำงานของ Network Application และ Application-Layer Protocol มากกว่าการนำไปใช้งานจริง

การสื่อสารยังไม่มีเข้ารหัส

ข้อมูลที่ส่งผ่าน TCP ในระบบปัจจุบันเป็นข้อมูล Text ที่ไม่ได้เข้ารหัส ดังนั้นข้อมูลที่ส่งระหว่าง Client และ Server สามารถมองเห็นได้หากมีการดักจับ Traffic ภายใน Network

ข้อจำกัดนี้สามารถพัฒนาเพิ่มเติมได้ในอนาคต เช่น การใช้ TLS เพื่อเพิ่มความปลอดภัยในการรับส่งข้อมูล

Server เป็นศูนย์กลางของการสื่อสาร

Client ทุกเครื่องต้องเชื่อมต่อกับ Server เพื่อส่งและรับข้อความ หาก Server หยุดทำงาน Client จะไม่สามารถสื่อสารกับผู้อื่นได้

ลักษณะการทำงานจึงเป็นแบบ Client -> Server -> Client โดย Server ทำหน้าที่ตรวจสอบ Request จัดการรายชื่อผู้ใช้งาน และส่งข้อความไปยังผู้รับ

จำนวนผู้ใช้งานขึ้นอยู่กับทรัพยากรของ Server

Server ใช้ Thread แยกสำหรับจัดการ Client แต่ละ Connection ดังนั้นหากมีผู้ใช้งานจำนวนมาก อาจใช้ทรัพยากรของระบบเพิ่มขึ้น เช่น CPU และ Memory และอาจจำเป็นต้องปรับปรุงรูปแบบการจัดการ Connection ในอนาคต

สรุป

โครงการนี้เป็นการพัฒนา AppChat ซึ่งเป็น Network Application สำหรับการสื่อสารข้อความระหว่างผู้ใช้งาน โดยใช้รูปแบบการทำงานแบบ Client-Server Architecture ผู้ใช้งานสามารถเข้าสู่ระบบ ตรวจสอบรายชื่อผู้ใช้งานที่ Online ส่งข้อความแบบส่วนตัว ส่งข้อความแบบ Broadcast และออกจากระบบได้

ระบบเลือกใช้ TCP (Transmission Control Protocol) เป็น Transport Layer เนื่องจากการสนทนาผ่าน Chat ต้องการให้ข้อมูลที่ส่งถึงปลายทางอย่างถูกต้องและอยู่ในลำดับที่ถูกต้อง TCP มีคุณสมบัติในการตรวจสอบการส่งข้อมูลและจัดการการส่งข้อมูลซ้ำหรือข้อมูลที่สูญหาย ซึ่งเหมาะสมกับลักษณะของ Network Application ประเภท Chat

สำหรับ Application Layer ได้ออกแบบ Protocol ของตนเองชื่อ NCP (Network Chat Protocol) เพื่อกำหนดรูปแบบการสื่อสารระหว่าง Client และ Server โดย Protocol ประกอบด้วย Request หลัก ได้แก่ LOGIN, LIST, SEND, SENDALL, QUIT และกำหนด Response ที่ประกอบด้วย Status Code และ Status Phrase เพื่อให้ Client สามารถตรวจสอบผลลัพธ์ของ Request ได้อย่างชัดเจน เช่น 200 OK, 400 BAD_REQUEST, 401 UNAUTHORIZED, 404 NOT_FOUND, 409 CONFLICT, 500 SERVER_ERROR

นอกจากนี้ NCP ยังมีการกำหนด Message ID สำหรับข้อความแต่ละข้อความ เช่น MSG-0001, MSG-0002, MSG-0003 เพื่อช่วยให้สามารถระบุข้อความแต่ละรายการได้อย่างชัดเจน และรองรับการส่งข้อความทั้งแบบ One-to-One ผ่านคำสั่ง SEND และแบบ One-to-Many (Broadcast) ผ่านคำสั่ง SENDALL

ระบบยังมีการแสดง Request และ Response ที่ถูกส่งและรับในฝั่ง Client และ Server ทำให้สามารถตรวจสอบลำดับการทำงานของ Protocol ได้ รวมถึงสามารถใช้ Wireshark ในการตรวจสอบ Network Traffic เพื่อแสดงให้เห็นการรับส่งข้อมูลของ NCP ผ่าน TCP ได้

โดยรวมแล้ว โครงงานนี้แสดงให้เห็นกระบวนการพัฒนา Network Application ตั้งแต่การออกแบบ Client-Server Architecture การเลือก Transport Layer การออกแบบ Application-Layer Protocol การกำหนด Request/Response Message และการนำ Protocol ที่ออกแบบไปพัฒนาเป็นโปรแกรมที่สามารถสื่อสารระหว่างผู้ใช้งานหลายคนได้จริง

ในอนาคตสามารถพัฒนาระบบเพิ่มเติมได้ เช่น การเพิ่มระบบ Authentication การเข้ารหัสข้อมูล การจัดเก็บประวัติ การสนทนาใน Database การส่งไฟล์และรูปภาพ รวมถึงการปรับปรุง Server ให้รองรับผู้ใช้งานจำนวนมากได้อย่างมีประสิทธิภาพมากขึ้น